smell-agent: step-by-step guide

Built from the "Teaching an AI to smell" guide, set up for Jupyter on Linux inside the `smell-agent` folder.

Before you start

- You need Python 3.10. OpenPOM's README is written for 3.10.
- An NVIDIA GPU makes training faster but is not required. Steps 1 and 2 show both the GPU and CPU installs.
- The sensor half (Part B) is optional. Do Part A first.

Part A: the model

Step 1. Folder and environment

In a terminal:

```
cd smell-agent
python3.10 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
```

If you prefer conda, `conda create -n smell-agent python=3.10` and `conda activate smell-agent` do the same job.

Step 2. Install the libraries

The order matters: DeepChem, then PyTorch, then DGL, then OpenPOM. These versions come from the OpenPOM README.

With an NVIDIA GPU (CUDA 12.4):

```
pip install --pre deepchem
pip install torch==2.4.0 torchvision==0.19.0 torchaudio==2.4.0 --index-url
https://download.pytorch.org/whl/cu124
pip install dgl -f https://data.dgl.ai/wheels/torch-2.4/cu124/repo.html
pip install openpom
```

Without a GPU, swap the PyTorch and DGL lines for the CPU builds:

```
pip install torch==2.4.0 torchvision==0.19.0 torchaudio==2.4.0 --index-url
https://download.pytorch.org/whl/cpu
pip install dgl -f https://data.dgl.ai/wheels/torch-2.4/repo.html
```

The DGL CPU link is from memory, so if it fails, check DGL's install page for the torch 2.4 CPU wheel.

Then add the extras the training notebook uses, and Jupyter:

```
pip install pyrfume dgllife pytest ipykernel scikit-multilearn jupyter
python -m ipykernel install --user --name smell-agent
```

Step 3. Check the install

Start Jupyter from the `smell-agent` folder:

```
jupyter notebook
```

Create `01_setup_check.ipynb`, pick the `smell-agent` kernel, and run:

```
import torch
import dgl
import deepchem
import openpom

print(torch.__version__)
print(torch.cuda.is_available())
```

If the imports give no error, you are set. `False` on the last line just means you are on CPU.

Step 4. Get the data and example notebooks

The pip package does not include the example notebooks, so clone the repo next to your notebooks:

```
git clone https://github.com/ARY2260/openpom.git
```

You will use two things from it:

- the dataset:
  `openpom/openpom/data/curated_datasets/curated_GS_LF_merged_4983.csv`

- the training notebook: `openpom/examples/mpnn_pom_train.ipynb`

Look at the data first in a new notebook:

```
import pandas as pd

path = "openpom/openpom/data/curated_datasets/curated_GS_LF_merged_4983.csv"
df = pd.read_csv(path)
print(df.shape)
df.head()
```

You should see 4,983 rows: one SMILES column and one column per smell word.

Step 5. Count the labels

This shows the class imbalance the guide talks about:

```
counts = df.drop(columns=df.columns[:1]).sum().sort_values(ascending=False)
print(counts.head(10))
print(counts.tail(10))
```

If the first column is not the SMILES column in your copy, adjust the `drop` line.

Step 6. Train

Copy `mpnn_pom_train.ipynb` into your `smell-agent` folder and open it with the `smell-agent` kernel. Fix any file paths at the top so they point to the CSV above.

Read through it once before running. It does these things in order:

1. Loads the CSV and turns each SMILES into an atom graph (the featurizer).
2. Splits 80/10/10 into train, validation and test with the multi-label stratified splitter.
3. Weights each smell word so rare ones count.
4. Builds `MPNNPOMModel` (5 message passing rounds, 256-number odor map, dropout 0.12).
5. Trains and reports ROC-AUC.

The notebook trains only 5 epochs. Run it once as is to confirm everything works, then raise the number of epochs. The paper used 150. On CPU, expect it to be slow, so raise it in steps (for example 20, then 50).

Step 7. Read your score

The score is ROC-AUC on the test set, averaged over the words. 0.5 is guessing and 1.0 is perfect. Full training with an ensemble of 10 reaches about 0.89, so a single short run will land lower. That is normal.

Step 8. Predict from a molecule

`predict_odors.py` needs the 10-model ensemble in `./models/ensemble_models`. The repo has no trained models, so you train them with the ensemble notebooks in `examples`. This is the longest part, so leave it for when a single model works.

Things to know:

- Run it from the repo folder: `python predict_odors.py`.
- On CPU, edit the `device_name` setting in the script to `cpu`.
- It opens a small desktop window, so it needs a display. If the window fails to open on Linux, install tkinter (`sudo apt install python3-tk`).

Try `CCO` (ethanol) first, then a few molecules whose smell you already know.

Part B: the sensor (optional)

You need a Bosch BME688 breakout board (Adafruit sells one), a Raspberry Pi, and a Windows machine for BME AI-Studio. The Windows part is the only piece that does not run on Linux.

1. Run a new board for a day or two before recording. Warm it up about 30 minutes each session.
2. Record each smell several times in a sealed container on the Pi. Follow the PiCockpit tutorial "Teach your BME688 how to smell".
3. Pick smells you can repeat, such as coffee and plain air.
4. Import the recordings into BME AI-Studio on Windows, make one class per smell, and train.
5. Export the result to the board.
6. Test on recordings from a different day than the training ones. Humidity and sensor drift can fool you otherwise.

Above 80% accuracy on unseen recordings is worth putting on the board, and above 90% is good. A perfect score usually means too little data.

Common mistakes

- Looking for a trained model to download. There is none, you train it.
- Installing in the wrong order, or mixing torch and DGL versions.
- Judging the first 5-epoch run as the final result.
- Expecting the sensor to name a new smell. It picks the closest one it knows.
- Testing the sensor on recordings from the same sitting it trained on.

Suggested folder layout

```
smell-agent/
  .venv/
  openpom/            (cloned repo)
  01_setup_check.ipynb
  02_explore_data.ipynb
  03_train.ipynb
  models/
```